BVS1 – Betriebssysteme

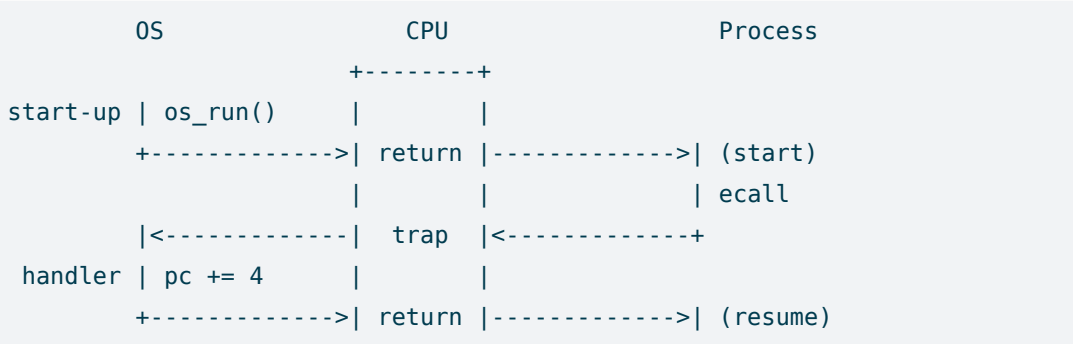
Lecture 6 – Timer and Preemption · Winter Semester 2026/2027

Stavros Kousidis

TH Köln · ICCT

Where We Stand

Recall: The Lifetime Through the CPU



Every trap so far was **caused by the process**: an `ecall`, or a faulty instruction.

The CPU can enter the OS. So far only the process decides when.

CC BY-SA 4.0

What If the Process Never Yields? Still Stuck

```
for (;;) { /* keep computing, never call */ }
```

Lecture 4 ended with this loop, Lecture 5 began with it. Since then our OS is entered by **traps**.

Why does this loop still never give control back to the OS?

CC BY-SA 4.0

Two Halves of the Answer: Today the Second

```
Lecture 5: the process asks       ecall   -> CPU -> OS
Lecture 6: an event interrupts    timer   -> CPU -> OS
```

Time passes whether the process cooperates or not.

Today's question: how does the OS get control back **without** the process's cooperation?

Today's plan: a device that keeps time and raises an interrupt request at a set time; the conditions under which the CPU accepts that request; and the trap path of Lecture 5, now entered by that request.

CC BY-SA 4.0

Time as an Event: The Platform Timer

Where Loads and Stores Go: the Address Map of QEMU `virt`

Some addresses lead to a device, not to RAM. Ordinary loads and stores reach it.

```
address range            what answers   role
0x02000000 - 0x0200ffff  CLINT          keeps time (today)
0x10000000 - 0x100000ff  UART           serial output
0x80000000 - 0x87ffffff  RAM            program and data
```

QEMU's `virt` machine **provides** these devices at fixed addresses; our code only uses the addresses. RAM starts at `ORIGIN = 0x80000000` of the Lecture 2 linker script; `-m 128M` makes it 128 MiB. CSRs have no address at all.

The **CLINT** (core-local interruptor) keeps time and **requests** timer interrupts from the CPU.

A device keeps time; the CPU's trap rule reacts to its request.

CC BY-SA 4.0

Two Device Registers: `mtime` and `mtimecmp`

Register	Address	Holds
`mtime`	`0x0200bff8`	the time since reset, in counts: +1 every 100 ns
`mtimecmp`	`0x02004000`	the time of the next timer interrupt

10 MHz = 10,000,000 counts per second. The **CLINT** counts `mtime`, not the OS. It never stops; our OS never resets it.

Both are 64 bits wide and **memory-mapped**, like the UART: ordinary loads and stores reach them, not the `csr…` instructions.

When `mtime >= mtimecmp`, the CLINT requests a timer interrupt.

Reading and Writing a Device Register in C

From the provided `riscv.h`:

```
#define CLINT_MTIMECMP0 ((volatile uint64_t *)0x02004000UL)
#define CLINT_MTIME     ((volatile uint64_t *)0x0200bff8UL)
static inline void timer_arm(uint64_t delta)
{
    *CLINT_MTIMECMP0 = *CLINT_MTIME + delta;
}
```

- The cast makes the number an **address**; `*` loads or stores there.
- `uint64_t`: exactly 64 bits, as wide as the register.
- `volatile`: every access really happens, in order; the device may change the value.

`timer_arm(delta)` requests the next timer interrupt `delta` counts from now.

CC BY-SA 4.0

How Long Is a Time Slice?

A **time slice** is how long the OS lets a process run until the next timer interrupt.

`mtime` counts at `TIMER_HZ` = 10 MHz: 10,000,000 counts per second.

The OS wants a time slice of **10 ms**.

Which value must `mtimecmp` receive?

CC BY-SA 4.0

Pending: A Request, Not Yet a Trap

Example: `mtimecmp` = 100000.

`mtime`	99998	99999	100000	100001	100002	...
`pending`	no	no	yes	yes	yes	...

A timer interrupt is **pending** while `mtime >= mtimecmp`: the device requests it. Whether the CPU **takes** it is decided by the CPU (next section).

It stays pending until the OS writes a larger `mtimecmp`.

CC BY-SA 4.0

Letting the Interrupt In

Three Conditions for Taking a Timer Interrupt

Condition	Recorded in	Set by
pending	`mip.MTIP` (bit 7)	the CLINT, while `mtime >= mtimecmp`
timer switch on	`mie.MTIE` (bit 7)	the OS: `csr_set(mie, MIE_MTIE)`
global switch on	`mstatus.MIE` (bit 3)	the CPU's rules: trap and `mret`

When all three hold, the CPU applies the **trap rule of Lecture 5** before the next instruction:

```
record     mepc <- pc, mcause <- reason
off        mstatus: MPIE <- MIE, MIE <- 0
jump       pc <- mtvec
```

Always write `mie.MTIE` or `mstatus.MIE`, never `MIE` alone.

CC BY-SA 4.0

Who Sets Which Bit in Our System

```
void os_run(void)               /* start-up, once */
{
    csr_write(mtvec, (uintptr_t)trap_entry);
    timer_arm(TIME_SLICE);      /* first request in 10 ms */
    csr_set(mie, MIE_MTIE);     /* csrs: timer switch on */
    ...
    os_start(...);              /* mret: MIE <- MPIE = 1 */
}
```

During `os_run()`, the global switch `mstatus.MIE` is still 0 from reset. The interrupt can only be taken after `mret`, in the process.

The OS arms the timer and turns on the timer switch; `mret` turns on the global switch.

The Global Switch Over Time: the Payoff of “Interrupts Off”

Recall (Lecture 5): the trap rule turns the global switch off, `mret` turns it on again.

```
process runs   mstatus.MIE = 1         timer may interrupt
trap           MPIE <- MIE, MIE <- 0
OS runs        mstatus.MIE = 0         a request waits
mret           MIE <- MPIE, so 1       timer may interrupt
```

A timer interrupt that becomes pending while the OS runs **waits**. It is taken right after `mret`, in the process.

Nothing interrupts the OS while it saves the process’s state.

In Exercise 4, Which Conditions Already Held?

The Exercise 4 library never wrote `mtimecmp` or `mie`.

Condition	In every Exercise 4 run, while the process ran?
pending: `mip.MTIP`	?
`mie.MTIE`	?
`mstatus.MIE`	?

Which conditions held, and why was the process never interrupted?

CC BY-SA 4.0

In Exercise 4: Exactly One Bit Was Missing

Condition	In Exercise 4, while the process ran
pending: `mip.MTIP`	**yes**: QEMU starts `mtimecmp` at 0, so `mtime >= mtimecmp` from reset
`mie.MTIE`	**0**: 0 at reset on QEMU, never set
`mstatus.MIE`	**1**: `os_start` sets `MPIE`, `mret` copies it

The timer switch alone kept the interrupt out.

The specification fixes only `mstatus.MIE` = 0 at reset; `mie` and `mtimecmp` are left open. **Never rely on reset values: arm the timer first, then turn on the timer switch.**

The Same Trap Path, Now Asynchronous

The CPU Takes the Interrupt: The Trap Rule of Lecture 5

Step of the trap rule	After `ecall`	After a timer interrupt
record where: `mepc`	the `ecall` itself	the next instruction, not executed
record why: `mcause`	11	interrupt bit set, code 7
interrupts off	`MPIE <- MIE`, `MIE <- 0`	the same
jump: `pc <- mtvec`	`trap_entry`	the same `trap_entry`

Nothing new in the CPU: the same rule, a different cause.

CC BY-SA 4.0

mcause: Interrupt Bit First

```
mcause = 0x8000000000000007
         bit 63 = 1: interrupt
         code   = 7: machine timer
```

mcause	Interrupt bit	Code	Meaning
0xb	0	11	ecall from M-mode
0x7	0	7	store/AMO access fault
0x8000000000000007	1	7	machine timer interrupt

The handler tests the interrupt bit first: code 7 alone is ambiguous.

The Trap `pc` Is the Continuation Address

Our test process for today, the **spinner**: a loop that counts and never yields.

```
while (counter < 1000000) { ++counter; }
```

As compiled (our build, GCC 13.2):

```
800002b4:  ld    a5,8(sp)       load counter
800002b6:  addi  a5,a5,1        +1
800002b8:  sd    a5,8(sp)       store counter
800002ba:  ld    a5,8(sp)
800002bc:  bgeu  a4,a5,800002b4 loop while counter <= limit
```

Timer interrupt reported at `0x800002b6`: `addi` has not executed yet.

After an interrupt, `mepc` already says where to continue. What goes wrong if the handler adds 4?

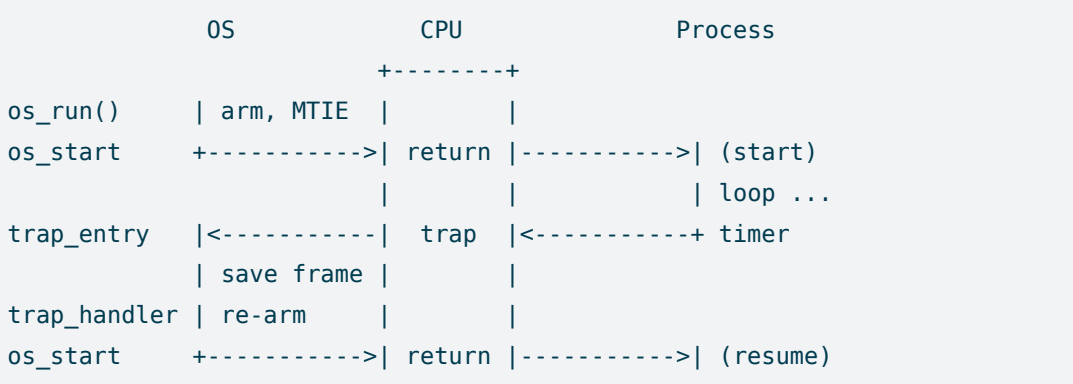
The Timer Branch of `trap_handler()`

```
uintptr_t cause = csr_read(mcause);
if (cause & MCAUSE_INTERRUPT) {          /* new branch */
    if (MCAUSE_CODE(cause) == IRQ_MACHINE_TIMER) {
        report_tick(trapped->pc);    /* Exercise 5 only */
        timer_arm(TIME_SLICE);       /* re-arm */
        return reschedule();         /* same frame, no +4 */
    }
} else if (cause == EXC_ECALL_FROM_M) {  /* Lecture 5 */
    trapped->pc += 4;  ...
}
...          /* unexpected trap: as in Lecture 5 */
```

What happens after **`mret`** if the handler forgets **`timer_arm()`**?

CC BY-SA 4.0

The Lifetime with a Timer Interrupt



The process never asked. The OS ran anyway, and returned the same frame.

Preemption

Preemption: Losing the CPU Without Yielding

Preemption: the OS takes the CPU away from a process that did not yield.

	Cooperative (Lecture 5)	Preemptive (today)
who causes the entry	the process: `ecall`	the timer
when	where the process chooses	between any two instructions
a `for (;;)` loop	keeps the CPU	loses it every 10 ms

From now on, no process can keep the OS out for longer than one time slice.

CC BY-SA 4.0

Interrupt Handling Versus the Scheduling Decision

```
interrupt handling     CPU: trap rule          the OS gets control
                       trap_entry: save        the state becomes data
scheduling decision    trap_handler(): which frame to return?
return                 os_start, mret          that frame continues
```

Today the decision is trivial: **one process, the same frame**.

Preemption makes a decision possible. It does not make one.

Demonstration: One Line per Tick

A **tick** is one timer interrupt of the OS.

```
spinner: start
os: tick 1 at 0x800002b8 in process 1
os: tick 2 at 0x800002b8 in process 1
...
os: tick 8 at 0x800002b4 in process 1
spinner: done, counter = 1000000
os: process 1 exited 7
os: all processes finished
os: 8 timer interrupts
```

Addresses and counts: our build (GCC 13.2, QEMU 8.2). Yours differ.

CC BY-SA 4.0

Can the Process Tell It Was Interrupted?

Same spinner, two libraries:

```
Exercise 4 library                Exercise 5 library
spinner: start                    spinner: start
                                  os: tick 1 ... os: tick 8
spinner: done,                    spinner: done,
  counter = 1000000                 counter = 1000000
```

Registers, memory and the next instruction are as before the interrupt.

The process cannot tell, except that time has passed.

CC BY-SA 4.0

Reproducible Interrupts with `-icount`

```
qemu-system-riscv64 ... -icount shift=4,sleep=off
```

- With `-icount`: virtual time advances by 16 ns (2^4 ns) per instruction. 10 ms = 625,000 instructions. Every run is interrupted at the **same** instructions.
- Without: QEMU follows host time. The places vary from run to run.

`-icount` is our observation tool, not a property of real hardware.

CC BY-SA 4.0

What Can Go Wrong, and the Whole Path

Diagnosis 1: Ticks Without Progress (More Diagnoses in Exercise 5)

A modified OS, same spinner:

```
os: starting 1 process
spinner: start
os: tick 1 at 0x800002b8 in process 1
os: tick 2 at 0x800002b8 in process 1
os: tick 3 at 0x800002b8 in process 1
...                      (tens of thousands of lines)
```

`spinner: done` never appears.

Which line of the timer branch is missing? Why is the address always the same?

CC BY-SA 4.0

Our Trap Model: Who Does What, with the Timer

Step	Who	What
request	CLINT	pending while `mtime >= mtimecmp`
trap	CPU	if enabled: `mepc`, `mcause`; interrupts off; `pc <- mtvec`
save	`trap_entry`	registers and trap `pc` into the frame
decide	`trap_handler()`	timer: re-arm, return a frame
restore	`os_start`	registers and `mepc` from the frame
return	CPU, `mret`	`pc <- mepc`; interrupts on

At CP2: your own `trap_handler()` and `os_run()`, with the timer (today) and a scheduler (Lecture 7).

Exercise 5: The Timer Path Against a Provided Library

Branch `exercise/05-timer`: the Exercise 5 `libkernel.a`, three broken libraries, `-icount` in `make run`. You add a `spinner()` to your Exercise 4 `main.c`.

Task	You do
1	predict, then run: with the Exercise 4 and the Exercise 5 library
2	find the reported `pc` values in `objdump`; explain the missing `+4`
3	the three conditions: where each is set (`os_run()`, `trap.S`, `riscv.h`)
4	compute the time slice in `mtime` counts
5	diagnose three broken libraries from their output

Same trap path, new cause: you write no handler code yet.

Key Ideas

- **CLINT:** `mtime` counts, `mtimecmp` sets the time of the next request.
- **Pending, enabled, taken:** `mip.MTIP`, `mie.MTIE`, `mstatus.MIE`.
- **Same trap path:** `mcause` with the interrupt bit; trap `pc` = continuation address.
- **Timer branch:** re-arm, then return a frame.
- **Preemption:** the OS gets the CPU back every time slice, without a yield.

Next: the OS has control at every tick. **Must it give the CPU back to the same process?**

CC BY-SA 4.0

References

Selected references:

- RISC-V International: *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture* — `mip`, `mie`, `mstatus`, machine timer.
- RISC-V International: *RISC-V Advanced Core Local Interruptor Specification* — `mtime`, `mtimecmp`.
- QEMU documentation: *TCG Instruction Counting* — `-icount`.
- Arpaci-Dusseau and Arpaci-Dusseau: *Operating Systems: Three Easy Pieces*, chapter *Mechanism: Limited Direct Execution*.

CC BY-SA 4.0